

Experimental Design of Self-Tracking Camera Gimbal Robot Based on OpenCV

Di Zhao^{12*}, Yuhong Du³, Zhenyu Ding⁴, Bo Yuan⁵ and Yuxiang Wang⁶

Abstract—The emergence of COVID-19 has reduced the opportunities for offline meetings, making people's work and study more transfer to the internet platform. However, the viewing angle and distance of the camera cannot be considered both. Therefore, machine vision is used to identify and track the presenter, and the camera pan-tilt control function of automatically tracking the presenter is realized. In many tests, the target tracking function works normally and works well. The experimental design involves relatively comprehensive disciplines, with good functional scalability and high practicability. It is an innovative experiment integrating robotics teaching, machine learning practice and embedded systems.

Keywords: Machine vision; Target identification and tracking; Micro Python; OpenCV; Dynamic tracking; Raspberry; YOLO

I Introduction

Machine vision is to simulate the human visual perception function through a microcontroller^[3]. It can convert the tracked target into an image signal and transmit it to the image processor, and then further convert it into digital signal according to the pixel distribution and pixel number, brightness, color and other information^[2]. This article put the target recognition system on the development board Raspberry Pi 4B, and a camera pan-tilt control robot based on the vision library OpenCV and MobileNet-SSD that can automatically track the speaker was designed^[13]. The gimbal can be plugged into the USB purchased by the user. It is convenient to connect with user equipment and has practical value

.II OVERALL SYSTEM DESIGN

This system uses Raspberry Pi 4B as the main platform, based on the Linux operating system, the CPU running frequency is up to 1.5GHz, and the platform chip is the 64-bit BCM2711, which runs stably and has low power consumption, which can meet the image processing requirements of the work^[6,9]. The image acquisition equipment uses a 5-megapixel camera that supports 100° distortion-free zoom, which can clearly complete real-time image acquisition. The Raspberry Pi 4B development board is connected to the servo through the GPIO interface, and the PID algorithm is used to control the rotation of the two-degree-of-freedom servo gimbal to drive the computer USB camera and the Raspberry Pi camera, so that the Raspberry Pi camera can capture video streams within a range of 360°, provides a basis for human target recognition, and can drive the USB camera to output the speaker image to the user's computer. The system hardware design block diagram is shown in Fig. 1. And FIG. 2 shows the structure of the pylon.

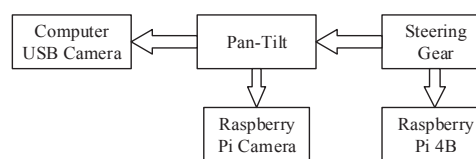


Fig.1 System hardware design block diagram

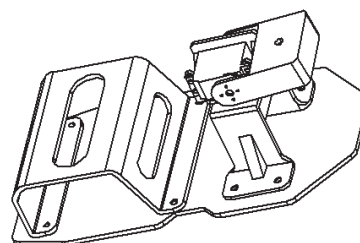


Fig.2 PTZ structure diagram

III DESIGN OF HUMAN BODY RECOGNITION SYSTEM BASED ON YOLO ALGORITHM

A. Overall System Architecture

The human body recognition system designed in this work is mainly composed of two parts: image and video acquisition and target information recognition^[8,11]. The overall framework of the system is shown in Fig. 3^[10].

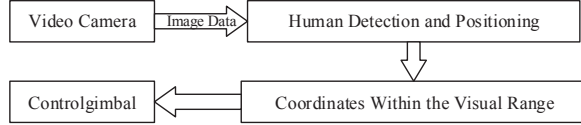


Fig.3 Overall system architecture

B. Principle of human body detection and location based on regression method

In recent years, the target detection algorithms represented by SSD and YOLO have become hot spots due to their fast recognition speed and high efficiency.

① SSD algorithm. The SSD target detection algorithm is a one-stage target detection method proposed by Wei Liu et al. Compared with two-stage target detection methods such as RCNN and SPPNet, the one-stage detection method does not need to generate candidate regions and then perform target classification, but directly adopts an end-to-end method for target category prediction and boundary regression, with a simple structure and higher efficiency, only a small loss of accuracy. Fig.4 shows the structure of the SSD network model.

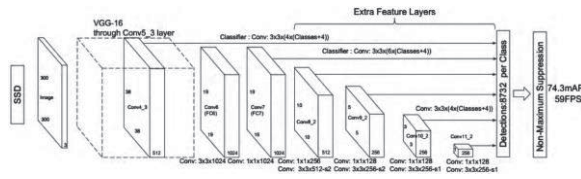


Fig.4 MobileNet-SSD model structure

The SSD model requires an input image of size $300 \times 300 \times 3$, and the detection framework consists of two parts: feature extraction network and multi-scale detection network. The front-end feature extraction network structure is VGG16, and multiple convolution layers are attached after VGG16 to make it used for feature extraction at different scales. In order to

achieve the task of target category recognition and location detection at the same time, the model selects six feature maps of different scales from multiple convolutional layers of the network structure to detect objects of different sizes. The input image is passed through VGG16 and the subsequent convolutional layers, then six feature maps of size $38 \times 38 \times 512$, $19 \times 19 \times 1024$, $10 \times 10 \times 512$, $5 \times 5 \times 256$, $3 \times 3 \times 256$, $1 \times 1 \times 256$ are generated. Then, different sizes and various aspect ratios are used in the six feature maps to generate a priori boxes, in which the probability of containing the object is predicted. The size of the prior box is generated by formula (1).

$$s_k = s_{\min} + \frac{s_{\max} - s_{\min}}{m-1} (k-1), k \in [1, m] \quad (1)$$

Where, m is the number of feature images, s_k is the ratio of the size of the candidate box to the size of the input image, and $s_{\max}$ and $s_{\min}$ are the maximum of the ratio to the example. Maximum value and minimum value. The aspect ratio of the prior frame of the box is selected from $\{1, 2, 3, 1/2, 1/3\}$. In the real training, it can adjust the length and width of the frame according to the shape characteristics of the human body.

After that, position regression and softmax classification were carried out on the prior frame, and the final detection result was generated by non-maximum suppression method. SSD network loss function is composed of position loss function and confidence loss function, as shown in formula (2).

$$L(x, c, l, g) = \frac{1}{N} (L_{\text{conf}}(x, c) + \alpha L_{\text{loc}}(x, l, g)) \quad (2)$$

Where: N is the positive sample number of prior frame, c is the predicted value of category confidence, l is the predicted value of prediction frame position, g is the position parameter of real frame, α is the weight factor, $L_{\text{conf}}(x, c)$ is the classification loss function, $L_{\text{loc}}(x, l, g)$ is the position loss function, x is the matching value of pre-labeled frame and prior frame, $x \in [1, 0]$.

② YOLO algorithm. Before the birth of YOLO algorithm, more common algorithms include R-CNN, Faster R-CNN and so on. These algorithms first do a lot of calculation on the possible target area, and then

use the classifier to detect each candidate box, give the confidence of each object, and realize the target recognition and location. Therefore, this kind of algorithm is also called candidate region based algorithm, which is usually very large. With the birth of YOLO algorithm, the convolution neural network algorithm is used to recognize the whole image with new ideas, and the recognition task is regarded as a regression problem. The detection speed and accuracy are improved [12]. Its detection process is shown in Figure 5.

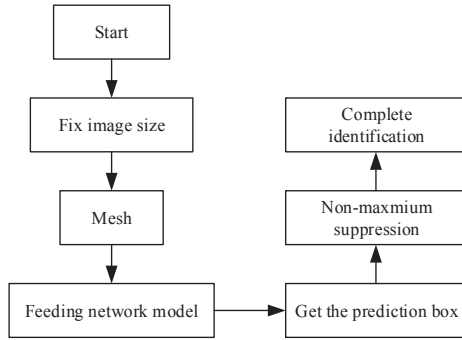


Fig.5 YOLO algorithm flow diagram

Formula (3) is the confidence formula, which is used to judge whether the detection target is in the prediction box and whether the prediction box is accurate.

$$confidence = Pr(Object) * IOU_{pred}^{truth} \quad (3)$$

Where, $Pr(Object)$ indicates whether the target to be predicted is in the prediction box. In the prediction box, the value is 1; otherwise, the value is 0. IOU represents whether the prediction box is accurate, which can be obtained by dividing the intersection and union of the predicted area and the target area. As shown in Formula (4).

$$IOU_{pred}^{truth} = \frac{Pred\ Area \cap Truth\ Area}{Pred\ Area \cup Truth\ Area} \quad (4)$$

For the target in the grid, YOLO algorithm will calculate five values, namely four coordinates (t_x, t_y, t_w, t_h) and a confidence t_0 . (t_x, t_y) is the center point coordinates of the predicted target, and (t_w, t_h) is the width and height of the target relative to the grid. In addition, the actual location and size information of the target are expressed by (b_x, b_y, b_w, b_h) , as shown

in Formula (5).

$$\begin{cases} b_x = \sigma(t_x) + c_x \\ b_y = \sigma(t_y) + c_y \\ b_w = p_w e^{t_w} \\ b_h = p_h e^{t_h} \end{cases} \quad (5)$$

Where, (c_x, c_y) represents the offset of the upper-left corner of the grid relative to the upper-left corner of the image; p_w and p_h represent the width and height of the prediction box respectively; σ represents the Sigmoid activation function; $\sigma(t_x)$ and $\sigma(t_y)$ represent the horizontal and vertical offset of the target position relative to the grid position after normalization respectively.

Assuming that the number of target types to be identified is N , the accuracy of each grid is $Pr(Class_i|Object)$, and the confidence of the target type can be obtained by multiplying the accuracy by the confidence. The calculation result includes not only the confidence probability of this type, but also the accuracy of the frame selection. Its calculation formula is shown in (6).

$$\frac{Pr(Class_i) * IOU_{pred}^{truth}}{Pr(Object) * IOU_{pred}^{truth}} = Pr(Class_i|Object) * \quad (6)$$

The detection target of this study is the teacher who is lecturing. The difference between the target and the background is obvious, and the target is large and the moving speed is slow. Therefore, priority is given to the detection speed of the system, followed by its detection accuracy. In this case, the YOLO algorithm is more suitable for this study. It is an end-to-end target recognition algorithm that can identify targets in a very short time. In addition, based on the regression idea, the complexity of the model is greatly reduced.

③ The improved YOLO algorithm. Most algorithms use the variance function as a loss function, and the same is true for the YOLO algorithm. Assuming that the training set contains N samples and the number of types is M , in order to make the input of an x value to the YOLO model be similar to the output y value, the optimal bias and weight are required. The loss function formula is shown in (7).

$$E^N = \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^M (t_m^n - y_m^n)^2 \quad (7)$$

The principle of algorithm optimization is to set the input of the algorithm as x and the output as y , and observe the change of the loss value through formula (7) to determine whether the optimization is completed. When the loss value area reaches a stable state, the optimization is completed.

In the YOLO algorithm, the target area only occupies a small part of the image, and most of the areas are backgrounds that need to be removed. Therefore, the loss function of the YOLO algorithm has two parameters, λ_{coord} and λ_{noobj} , for the confidence of the type and the prediction frame. The position is weighted to avoid the problem of insensitive target recognition. The modified loss function is shown in (8).

$$\begin{aligned} & \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B I_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \\ & \hat{y}_i)^2] + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B I_{ij}^{obj} \left[\left(\frac{\omega_i - \hat{\omega}_i}{\omega_i} \right)^2 + \right. \\ & \left. \left(\frac{h_i - \hat{h}_i}{h_i} \right)^2 \right] + \sum_{i=0}^{S^2} \sum_{j=0}^B I_{ij}^{obj} (C_i - \\ & \hat{C}_i)^2 + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B I_{ij}^{noobj} (C_i - \hat{C}_i)^2 + \\ & \sum_{i=0}^{S^2} I_{ij}^{obj} \sum_{c \in classes} (p_i(c) - \hat{p}_i(c))^2 \end{aligned} \quad (8)$$

In formula (8), the normalization content is added to the original loss function, so that the change of the large target and the small target will not have too much influence on the recognition result, which improves the problem that YOLO recognition is not sensitive enough, and improves the recognition accuracy.

IV DESIGN OF PTZ CONTROL SYSTEM WITH TWO DEGREES OF FREEDOM

The mechanical mechanism of the two-degree-of-freedom gimbal is shown in Fig. 6, and its degrees of freedom include two directions, namely the pitch angle and the horizontal angle^[1]. In addition, it is also equipped with a USB camera for the user's computer to use.

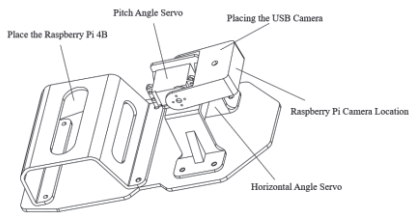


Fig.6 Mechanical structure of 2-DOF PTZ

The program block diagram of the PTZ control system is shown in Figure 7. First initialize some peripheral hardware and controllers, and then enter the main loop of the timer. Control the gimbal. The control system of the PTZ is divided into 3 modes, namely tracking mode, remote control mode and standby mode. In the tracking mode, the Raspberry Pi receives the target position for image processing; in the remote control mode, the Raspberry Pi processes the remote control information; in the standby mode, it keeps stopping.

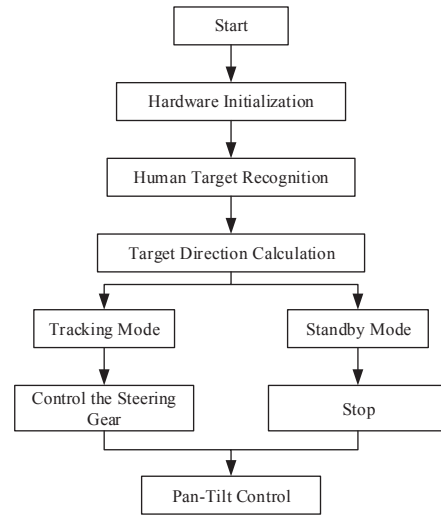


Fig.7 PTZ control program block diagram
V System debugging

A. Target recognition debugging

The purpose of this experiment is to find the position of the human body through OpenCV, and give the coordinates of the human body in the video and the number of frames processed in real time.

(1) PC-side code debugging

Lab environment: Windows 10、Python 3.8.10、OpenCV-python 4.5.3.56、Numpy 1.21.2

Experimental software: Pycharm 2021.3

Call the OpenCV and Numpy libraries in Pycharm on the PC side, and first obtain the camera image with a width and height of 640×480. The OpenCV 4.5.3.56 version used by scholars provides three usable classes for the deep neural network module, which can easily pre-process and post-process the input image. Here, the `cv2.dnn_DetectionModel` class was used to load the model, and then

cv2.dnn_DetectionModel was used to load the model's config file and weight file. Then the input image was specified to be scaled to 160 wide and 120 high, normalize the input data and exchange the RB channel data. After completing the above initialization, use the detect function to detect the input image, set the confidence threshold confThreshold of the filter selection box to 0.61, the function will return a tuple, including (classIds, confidences, boxes), where "classIds" is the category index, 1 means human body is recognized; 0 means no human body is recognized; "confidences" is the confidence level; "box" is the coordinates of the detection frame, and the values of the box are the starting abscissa x, the starting ordinate y, "width" means the width of detection frame, and "height" means the height of the detection frame.

Output the coordinates of the box detection frame, find the center point in the detection frame, and use the putText function to mark the center point. Output the offset of the center point of the human body to the center of the image. Figure 8 shows the experimental output results.

```
fps : 35.308858564345186
[ 92  72 366 408]
X-axis offset: -45.0
Y-axis offset: 36.0
fps : 35.308858564345186
[ 90  72 366 408]
X-axis offset: -47.0
Y-axis offset: 36.0
```

Fig.8 Experimental output results on PC

It can be seen from the output results that the frame rate of real-time target recognition is about 33~35 frames per second.

(2) Porting to Raspberry Pi platform for debugging

The output result after transplanting the above program is shown in Figure 9.

```
[163 117  97 101]
X-axis offset: -108.5
Y-axis offset: -72.5
fps : 8.751826294890547
[163 117  97 101]
X-axis offset: -108.5
Y-axis offset: -72.5
fps : 8.604741907225694
```

Fig.9 Experimental output results on Raspberry Pi

Obviously, the number of frames on the Raspberry Pi platform is far lower than the experimental results on the PC side. The reason for this is that the performance of the Raspberry Pi CPU is lower than that of the PC side, so the real-time processing speed and frame rate are lower than those of the PC side. In order to solve this problem, the acquired image data on the Raspberry Pi platform is not processed per frame, but frame-by-frame processing.

By properly judging the offset, the offset of the position of the human body in the entire screen can be obtained, that is, the target direction of rotation can be obtained^[7]. In the program, the horizontal target is left or right, and when the adjustment is completed, it is X-axis ok; in the same way, the vertical target is up or down, and when the adjustment is completed, it is Y-axis ok. Figure 10 shows the target output result.

```
[ 3  55 349 425]
left
Y-axis ok
fps : 22.259451886131572
[ 2  57 356 423]
X-axis ok
Y-axis ok
fps : 22.754814838926684
```

Fig.10 Target output result

B. PTZ control debugging

Servo model: MG995(360degree)、MG995 (180 degree).

(1) Initial position calibration

The horizontal angle steering gear used in the system is a 360° steering gear, and the pitch angle steering gear is a 180° steering gear. The control of the steering gear requires a time base pulse of about 20ms. The duty cycle of the control signal is in one-to-one correspondence with the angle of the 180° steering

gear. The corresponding relationship is shown in Table 1. Therefore, it is necessary to set the position of the steering gear corresponding to the signal with a duty cycle of 50% to the front of the system. Since the 360° servo will only turn left or right according to the size of the PWM, there is no need to calibrate the initial position.

Table.1 high level time corresponding to steering gear angle

High time	Servo angle
0.5ms	0
1.0ms	45
1.5ms	90
2.0ms	135
2.5ms	180

(2) Control

Connect the brown wire (or black wire) of the servo to the GND of the Raspberry Pi, the red wire of the servo to the 5V of the Raspberry Pi, and the yellow wire of the servo to any GPIO output port of the Raspberry Pi. On the Raspberry Pi platform, the RPi.GPIO library in python can be used to control the IO port, and the rotation of the servo can be controlled by software PWM, and it can be rotated to X-axis ok and Y-axis ok according to the result of human body recognition.

rgt output results.

VI RESULTS

In this paper, a self-tracking camera gimbal control robot was designed and experiments were carried out, and the USB camera purchased by the user can be plugged in, which can be applied to the real online meeting scene, and has strong practicability. This paper expounds the design idea of the system from the aspects of human body detection and positioning and PTZ control, using machine vision to obtain the coordinates of the human target in the perspective, and then controlling the steering gear to achieve the goal of tracking. In the near future, the detection speed, detection accuracy, and recognition accuracy of the final algorithm can be improved by improving the model structure and the process of image preprocessing.

REFERENCES

- [1] Shuo Zhu. Design of target recognition and tracking system for unmanned platform based on machine vision[D]. Nanjing University of Science and Technology,2018.
- [2] Xueli Zhang,Yazhou,Zhang,Kun Fang. Research on rolling target Recognition and Tracking system of OpenMV Vision Module[J]. Microcontroller and Embedded System Application,2021,21(02):8-10.
- [3] Shiwei Zhou, Jiayuan Gong, Leipeng Qie, et al. Design of PTZ tracking system based on OpenMV[J]. Journal of Hubei Institute of Automotive Technology,2019,33(04):54-56.
- [4] HOWARD A G,ZHU M,CHEN B,et al. MobileNets:Efficient Convolutional Neural Networks for Mobile Vision Applications[EB/OL].(2017-04-14)[2019-06-23].
- [5] LI Y,HUANG H,XIE Q,et al. Research on a Surface Defect Detection Algorithm Based on MobileNetSSD[J]. Appl Sci,2018,8(9):1678.
- [6] Lei Hua, Xiaohui Yuan. A fast positioning method of camera head based on image[J]. Instrumentation and Analytical Monitoring,2009(03):15-17.
- [7] Pengfei Sun, Xiqing Guo, Jingxina Yang. Machine vision control of two degrees of freedom head[J]. Photoelectric Engineering. ,2012,39(10):135-142.
- [8] Jiong Zhao, Zhengdong Xie, Qicai Zhou, et al. Research on design and control method of 2 dOF PTZ control system[J]. Mechanical Engineering and Automation,2020(06):21-24.
- [9] Liying Shi. Multi-target recognition and position estimation of aircraft based on visible light images [D]. Shenyangligong University,2020.
- [10] Bochen Zhang, Xinjie Shi,Meimei Huo. Raspberry PI face recognition fatigue driving detection system based on OpenCV[J]. Modern Computer,2021,27(23):129-132.
- [11] Kai Zhu. Research on Armored Vehicle Recognition Technology Based on Deep Learning[D]. Nanjing University of Science & Technology,2019.
- [12] Yinggang He, Xinwang Zheng, Jiakun Ruan. Design and implementation of License plate Detection and Recognition System based on Mobilenet-SSD and BI-GRU[J]. Journal of Tonghua Normal University,2021,42(08):17-24.
- [13] Wei Wang, Yanlong Zhang, Denghui Zhai, et al. Intelligent identification of substation platen state based on OpenCV+SSD deep learning model[J/OL]. Electrical Measurement and Instrumentation:1-10[2021-10-14].